

ENGINEERING · DISTRIBUTED SYSTEMS · FINANCIAL INFRASTRUCTURE**Designing Financial Infrastructure for Extreme Scale (50K+ req/sec)**

A systematic analysis of where a 100M events/day fraud detection pipeline breaks at JPMC and PayPal throughput levels — and the architectural changes required at each scale inflection point.

Utkarsh Mehrotra · Chief Engineer, Samsung Electronics · Ex-Western Union

BASELINE (BUILT)	PAYPAL TIER	JPMC TIER
1,150/s 100M events/day <1s P99 latency 3 stream types Single region	~1,500/s ~40M txns/day + aux <300ms P99 scoring 10+ stream types Multi-region active-active	50,000+/s \$10T+ annually <50ms P99 on card rails Microsecond ACH/RTGS SLAs Global multi-datacenter

Introduction

The fraud detection pipeline described in a [previous post](#) was designed to solve a specific organisational problem: giving a fraud analytics team the ability to act on attack patterns in near real-time rather than across sprint cycles. At 100M events/day and sub-second latency, it does that reliably. But the same architectural decisions that make it correct at that scale

would cause it to fail quietly — and then loudly — under the throughput requirements of a JPMC or PayPal.

The failure is not hypothetical. Tier-1 financial institutions have learned these lessons expensively: JPMC's payment infrastructure must sustain fraud scoring at 50,000+ events/second with a 50ms P99 SLA on card rails. A single architectural assumption that holds at 1,150 events/second — Flink managing all session state in RocksDB, a single model handling all fraud typologies, passive failover for regional outages — becomes a liability that manifests as missed fraud, regulatory exposure, or payment rail downtime at that throughput.

This post is a systematic analysis of exactly which components fail, at which scale inflection, and why. For each failure mode, we describe the options we evaluated and the specific reasons for the architectural changes required. The goal is to be precise about the engineering tradeoffs rather than assert that scale requires "more infrastructure."

Where the Baseline Design Fails

The five structural failure modes that emerge as throughput scales beyond 1,000 events/second are distinct and require different remediation strategies:

LATENCY

Tail latency amplification. At 1,150 events/second, a P99 of 800ms is acceptable. At 50,000/second, tail latency compounds across stages — a single slow ML scoring call backs up the Spark rule evaluation stage, which backs up the Flink enrichment stage, which backs up Kafka consumer groups. A single end-to-end SLA must be replaced with per-stage latency budgets, each enforced independently with circuit breakers at every service boundary.

STATE SIZE

Flink RocksDB checkpoint degradation. Flink's RocksDB state backend begins exhibiting significant checkpoint overhead above approximately 100GB per task manager. At PayPal-scale user populations, checkpoint sizes balloon proportionally, and

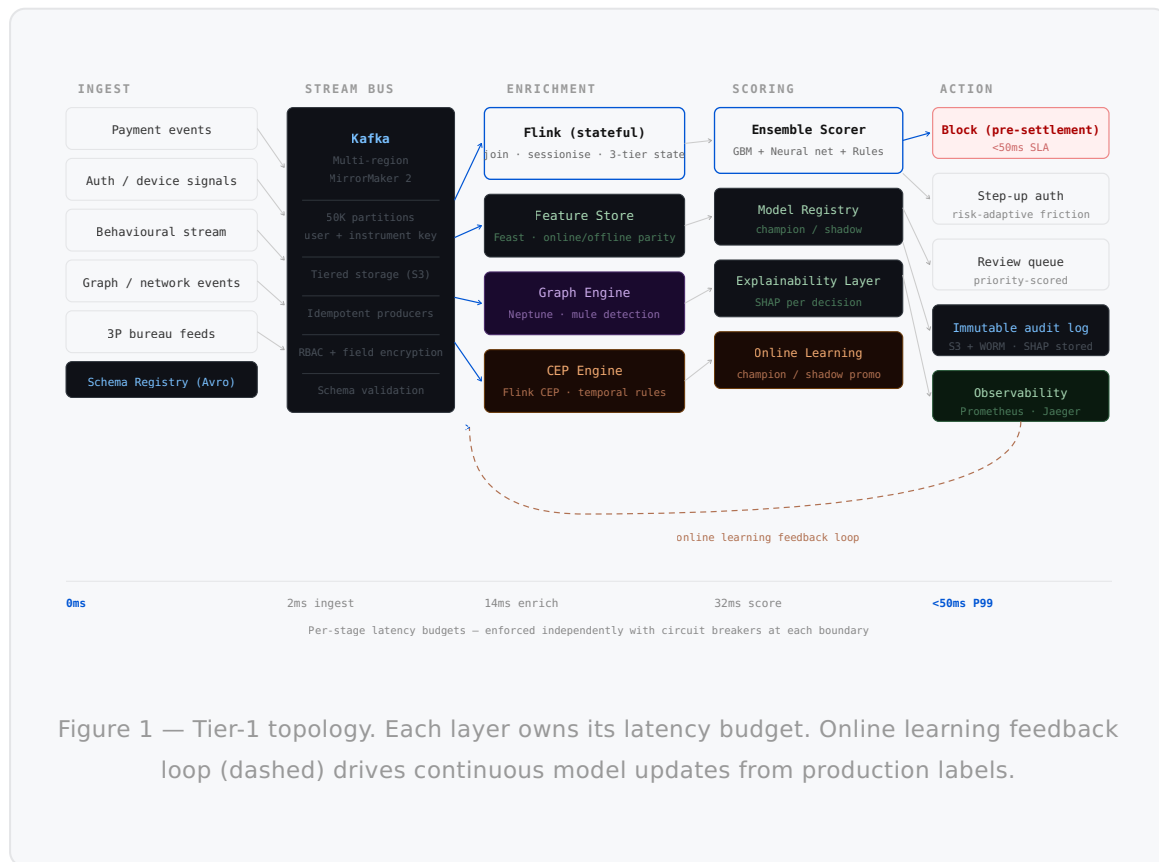
recovery time following a task manager failure extends beyond ten minutes. This is unacceptable when payment scoring must resume within seconds. The solution requires explicit state tiering: hot session context in JVM heap, 24-hour history in RocksDB, and long-term state in an external store.

REGULATOR **Explainability requirements.** At institutions operating under OCC, Federal Reserve, and international regulatory frameworks, every fraud decision must be reconstructable deterministically: the input feature vector, the model version in production at decision time, and the decision logic must all be retrievable from an immutable audit log. Sampling SHAP values or relying on log approximations is not compliant. This is infrastructure, not a post-hoc analytics requirement.

GEO-DISTRIBUTION **Single-region availability.** Active-passive failover at JPMC scale produces unacceptable recovery times. Leader election, connection rerouting, and state reconciliation after a regional failure takes 15–45 seconds at minimum — long enough to halt payment rails and cause material financial impact. Active-active multi-region operation is required from the outset, which introduces distributed state conflict resolution as a first-class engineering problem.

MODEL FRESHNESS **Batch retraining cadence.** Fraud attack patterns shift within hours during an active campaign. A model trained on prior-day data may be blind to a new mule-network variant or synthetic identity pattern that emerged that morning. Tier-1 financial institutions require continuous model update mechanisms — online learning or near-continuous retraining pipelines — that push updated scoring logic to production infrastructure within minutes, not days.

Redesigned Architecture



Component-Level Changes

INGESTION: SCHEMA ENFORCEMENT AND COMPOUND PARTITION KEYS

At 1,150 events/second, a schema-incompatible event from a misbehaving producer causes a Flink job exception that surfaces quickly in monitoring. At 50,000 events/second, the same producer firing for 30 seconds before detection can corrupt checkpoint state across multiple task managers and require a full state rebuild.

We address this by enforcing Avro schemas through Confluent Schema Registry at the producer boundary. Schema-incompatible events are rejected before reaching Kafka — the failure is loud and immediate at the producer, rather than silent and cascading inside the pipeline.

Additionally, the partition key changes from `user_id` to `user_id +`

`instrument_id` : at PayPal-scale user populations, a single user may hold dozens of payment instruments, and per-instrument velocity tracking across partitions requires expensive coordination that a compound key eliminates.

STATE MANAGEMENT: THREE-TIER ARCHITECTURE

The default Flink RocksDB state backend begins degrading at approximately 100GB per task manager. At PayPal-scale, state sizes routinely exceed this threshold, checkpoint durations extend, and post-failure recovery times cross ten minutes — long enough to produce measurable gaps in fraud scoring coverage.

The redesigned state architecture uses explicit tiering based on access recency:

- **JVM heap** — last 60 seconds of per-user session context. Microsecond access. Used for in-flight event enrichment.
- **RocksDB** — 24-hour feature history. Low-millisecond access. Used for velocity calculations and session reconstruction.
- **Redis Cluster / Aerospike** — longer-horizon features and cross-session history. Low-millisecond remote access. Used for ML feature retrieval.

Recovery time under this model is bounded by the RocksDB checkpoint size rather than total state size, since hot state is rebuilt from events after restart.

FEATURE STORE: ELIMINATING TRAINING/SERVING SKEW

In the baseline architecture, features are computed inside the Flink job. This creates a structural coupling that becomes a significant problem at scale: the feature logic used in offline model training diverges from the logic applied at serving time. This gap — training/serving skew — is one of the most common sources of unexplained model degradation in production ML systems and one of the hardest to diagnose.

A dedicated feature store (Feast or Tecton) addresses this by providing a single feature definition that is executed identically by both the batch training pipeline and the real-time serving path. At tier-1 scale, where dozens of models consume hundreds of features, this is load-bearing infrastructure. Point-in-time correct feature retrieval — ensuring that training on historical data uses only features that were available at that moment — is a correctness requirement, not an optimization.

GRAPH ENGINE: NETWORK-LEVEL FRAUD DETECTION

Per-transaction scoring cannot detect coordinated fraud rings, where individual transactions appear legitimate but the account relationship graph reveals a money-mule network. At JPMC scale, this graph contains billions of edges and cannot be traversed naively at scoring time.

The solution is to pre-compute graph features on a scheduled basis and surface them through the feature store:

```
// 3-hop mule proximity feature — computed offline, served online
g.V().has('accountId', txn.senderId)
  .repeat(out('transferred_to').simplePath())
  .times(3)
  .where(has('flagged', true))
  .count()
  .next() // → ring_proximity_score fed into ensemble scorer
```

The real-time scoring path performs a feature lookup — not a live traversal — keeping graph-based signals on the critical latency path without incurring graph query latency at runtime.

SCORING: ENSEMBLE MODELS AND PER-DECISION EXPLAINABILITY

No single model architecture dominates across all fraud typologies. Gradient-boosted trees (XGBoost/LightGBM) perform well on velocity-based patterns. Neural networks better capture device fingerprint anomalies. Rule engines fire deterministically on known fraud signatures

with zero model latency. An ensemble meta-scorer, weighted by each model's recent per-category precision, combines these signals and provides redundancy: if one model degrades silently, the others continue to catch the majority of fraud.

SHAP values must be computed for every scored event — not sampled. Under OCC examination, a disputed fraud decision requires deterministic reconstruction from the logged feature vector, the model version hash, the SHAP attribution values, and the final score. Approximations are not compliant. All of this is written to an append-only S3 log with WORM retention policy before the action layer is invoked.

At this scale, the difference between 94% and 96% model precision is not two percentage points. It is tens of millions of dollars in avoided false-positive friction annually — legitimate transactions blocked, customers lost, support costs incurred. Ensemble scoring and continuous model evaluation are economics, not engineering refinement.

Latency Budget Model

The single most important operational discipline change at tier-1 scale is replacing a single end-to-end SLA with owned per-stage latency budgets, enforced with circuit breakers at each boundary. When a stage exceeds its budget, it triggers a circuit breaker rather than propagating delay downstream.

STAGE	P99 BUDGET	ENFORCEMENT MECHANISM
Kafka ingest	2ms	linger.ms=0, acks=1 for non-critical streams
Flink enrichment	12ms	Tiered state, async graph feature lookup, watermark tuning

STAGE	P99 BUDGET	ENFORCEMENT MECHANISM
Ensemble scoring	18ms	Parallel model calls, circuit breaker fires at 15ms
Action dispatch	5ms	Pre-warmed connection pools, local decision cache
Audit write	Async	Fire-and-forget to S3 — never on the critical path
Total P99	<50ms	End-to-end traced with Jaeger, SLO alert fires at 40ms

Moving the audit write off the critical path is the detail that surprises most engineers. At 50,000 events/second, a synchronous 5ms audit write contributes 250 seconds of cumulative blocking per second of pipeline throughput. Every non-decision output must be async.

Architectural Decision Log

Why active-active multi-region rather than active-passive with fast failover?

Recovery time arithmetic makes active-passive non-viable. We evaluated three options: active-passive with manual failover, active-passive with automated failover via a health-check controller, and active-active. Manual failover was eliminated immediately — human-in-the-loop recovery at JPMC-scale payment volumes is not a design, it is a disaster plan. Automated active-passive failover was the serious contender: it is operationally simpler and eliminates the distributed state conflict problem. The issue is arithmetic. At 50,000 events/second, the minimum automated failover time — Kubernetes health-check interval (30s default) plus leader election (5–15s) plus connection rerouting (10–20s) — produces a floor of roughly 45–65 seconds during which no fraud scoring occurs. At the transaction volumes of a major payment rail, 60 seconds of unscored traffic represents meaningful undetected fraud exposure. Active-active eliminates the outage by keeping

both regions serving live traffic continuously. The engineering cost — designing velocity counters and blacklists as CRDTs and deploying a globally replicated store — is the explicit price of the availability requirement.

Why an ensemble scorer rather than a single best-performing model?

No single architecture dominates across fraud typologies, and silent degradation is catastrophic. We evaluated three approaches: a single best-performing model (XGBoost, selected by offline F1 score), a model cascade (fast rule engine as a first pass, ML model for borderline cases), and a parallel ensemble. The single model performed best on the offline evaluation dataset — precisely the problem. Offline evaluation datasets reflect historical fraud patterns; they are systematically blind to emerging typologies. When the model degrades on a new attack pattern, there is no signal: precision drops on a category that was not prominent in the evaluation set, and the degradation is invisible until fraud losses accumulate. The cascade model was operationally attractive but introduced a hard dependency: if the rule engine's threshold is miscalibrated, it either over-filters (blocking ML scoring of legitimate borderline cases) or under-filters (passing everything to ML, negating the cascade benefit). The parallel ensemble, weighted by per-category precision on a rolling evaluation window, surfaces degradation immediately: a model's weight drops as its category-specific precision falls, and the other models compensate. The operational cost is three inference services rather than one. That cost is paid once; the benefit is continuous.

Why keep Flink and Spark as separate stages at this scale?

Operational specialisation under strict latency budgets. Flink's credit-based flow control and stateful join primitives are purpose-built for low-latency streaming. Spark's SQL engine and analytical execution model are better suited to complex rule evaluation across enriched micro-batches. At 50,000 events/second with a 50ms end-to-end budget, using a single engine for both means accepting its worst latency characteristics across the full pipeline. The operational overhead of two runtimes is the explicit tradeoff for maintaining distinct performance profiles per stage.

A Non-Obvious Engineering Problem: Distributed Velocity Counters Under Active-Active

The most technically demanding consequence of active-active multi-region operation is one that does not surface in architecture diagrams: how do you maintain accurate per-user transaction velocity counts when two regions are writing to the same logical counter simultaneously?

Velocity counters — "this user has initiated 7 transactions in the last 5 minutes" — are among the highest-signal fraud features. They are also exactly the kind of shared mutable state that active-active replication makes dangerous. A naive approach — replicate writes synchronously across regions — reintroduces the availability problem we solved by going active-active. Synchronous cross-region writes add 50–150ms of replication latency on every increment, which blows the 50ms total budget before any enrichment or scoring has occurred.

We designed velocity counters as G-Counters (Grow-Only CRDTs), with one logical counter implemented as a vector of per-region counts:

```
// G-Counter CRDT – each region maintains its own shard
// Global count = sum of all region shards
// No cross-region coordination required for increments

data class VelocityCounter(
    val regionCounts: Map<Region, Long> // {us-east: 4, eu-west: 3}
) {
    fun increment(region: Region) = copy(
        regionCounts = regionCounts + (region to (regionCounts[region] ?: 0L) + 1)
    )

    // Global velocity = sum of all region shards
    // Eventual consistency: may lag by replication delay (~50-200ms)
    val globalCount: Long get() = regionCounts.values.sum()

    // Merge is commutative, associative, idempotent – safe under re-delivery
    fun merge(other: VelocityCounter) = copy(
        regionCounts = (regionCounts.keys + other.regionCounts.keys)
```

```
        .associateWith { maxOf(regionCounts[it] ?: 0L, other.regionCounts[it] ?: 0L)
    }
}
```

Each region increments only its own shard — zero cross-region coordination. Regional shard values are asynchronously replicated on a 100–200ms interval. The global count used for fraud scoring is the sum of all shards, which may lag by up to one replication interval. This means a user who makes 5 rapid transactions split across two regions might be scored against a count of 4 for up to 200ms before the sixth increment replicates.

This is an intentional, bounded consistency relaxation. The alternative — strong consistency — requires cross-region coordination on every increment and reintroduces the latency problem we solved by going active-active. The fraud risk of a 200ms lag on velocity counting is substantially lower than the fraud risk of 60 seconds of unscored traffic during a regional failover. That tradeoff — precise consistency versus system availability — must be made explicitly and documented, not discovered during an incident.

The sliding window expiry adds a second layer of complexity: counters must decrement as events age out of the window. G-Counters are grow-only; they cannot natively support decrements. We handle this with a separate PN-Counter (Positive-Negative CRDT) for expiry events, with the effective count derived as increments minus expired-increments. The expiry events are generated by a per-region TTL sweeper that runs on a 10-second interval, ensuring the window boundary error is bounded and predictable rather than unbounded.

The upgrades described in this post are additive, not corrective. The three-layer topology (ingest → process → act), entity-keyed partitioning, asynchronous ML integration, externalised rule management, and dead-letter queue design from the baseline system are all valid and carry forward unchanged.

The components introduced at tier-1 scale — feature store, graph engine, ensemble scoring, tiered state, SHAP audit logging, active-active geo-distribution — are not improvements to a flawed design. They are the natural evolution of a sound design as throughput, regulatory complexity, and failure tolerance requirements increase. Each upgrade is justified by a specific failure mode that does not appear at 1,150 events/second but becomes load-bearing above 1,500.

Understanding which upgrade to apply at which scale inflection, and being able to articulate why each one is necessary at that point and not before, is the practical substance of distributed systems engineering at this level.